

Università degli Studi di Urbino Carlo Bo

Corso di Laurea in Informatica – Scienza e Tecnologia

Progetto di Programmazione Logica e Funzionale

Algebra dei Polinomi

Studenti:

Mattia Gasperoni

Matricola: 329235

Anno di corso: III

Alessio Cavalieri

Matricola: 327104

Anno di corso: III

Docente:

Prof. Marco Bernardo

Anno Accademico: 2025/2026

Indice

1	Specifica del Problema	2
2	Analisi del Problema	3
2.1	Dati di Ingresso del Problema	3
2.2	Dati di Uscita del Problema	3
2.3	Relazioni Intercorrenti tra i Dati del Problema	4
3	Progettazione dell'Algoritmo	7
3.1	Scelte di Progetto	7
3.2	Passi dell'Algoritmo	10
4	Implementazione dell'Algoritmo	12
4.1	Differenze Implementative	12
4.2	File sorgente Haskell (<code>operazioni_polinomi.hs</code>)	15
4.3	File sorgente Prolog (<code>operazioni_polinomi.pl</code>)	23
5	Testing del Programma	35
5.1	Testing del programma Haskell	35
5.2	Testing del programma Prolog	39

1 Specifica del Problema

Realizzare un programma in Haskell e un programma in Prolog che acquisiscano da tastiera due polinomi a coefficienti reali in una variabile, visualizzino i polinomi inseriti e ne calcolino il grado, la loro somma, la loro differenza, il loro prodotto, la divisione euclidea con resto e il loro massimo comune divisore.

2 Analisi del Problema

2.1 Dati di Ingresso del Problema

I dati di ingresso del problema sono due polinomi $P_A, P_B \in \mathbb{R}[x]$ a coefficienti reali in una variabile.

2.2 Dati di Uscita del Problema

I dati di uscita del problema consistono nei seguenti elementi, calcolati a partire dai polinomi in input:

- I polinomi P_A e P_B stessi, nella loro rappresentazione algebrica.
- I gradi $\deg(P_A)$ e $\deg(P_B)$.
- La somma $P_A + P_B$.
- La differenza $P_A - P_B$.
- Il prodotto $P_A \cdot P_B$.
- Il quoziente Q e il resto R della divisione euclidea di P_A per P_B (con $P_B \neq 0$).
- Il massimo comune divisore $\text{MCD}(P_A, P_B)$.

2.3 Relazioni Intercorrenti tra i Dati del Problema

Sia $\mathbb{R}[x]$ l'anello dei polinomi a coefficienti reali in una variabile. Un polinomio $P \in \mathbb{R}[x]$ è un'espressione della forma:

$$P(x) = \sum_{i=0}^n a_i x^i,$$

dove $a_0, a_1, \dots, a_n \in \mathbb{R}$ sono detti coefficienti del polinomio e a_i è il coefficiente del termine di grado i .

Se P è non nullo, il suo grado è definito come

$$\deg(P) = \max\{i \mid a_i \neq 0\}.$$

Il coefficiente corrispondente al termine di grado massimo è detto coefficiente direttore. Un polinomio non nullo è detto monico se il suo coefficiente direttore è uguale a 1. Il polinomio nullo è il polinomio i cui coefficienti sono tutti nulli; il suo grado non è matematicamente definito.

Siano ora $P_A, P_B \in \mathbb{R}[x]$ due polinomi non nulli di grado rispettivamente n e m , espressi nella forma

$$P_A(x) = \sum_{i=0}^n a_i x^i \quad \text{e} \quad P_B(x) = \sum_{i=0}^m b_i x^i.$$

Le operazioni richieste godono delle seguenti proprietà matematiche, rilevanti ai fini della risoluzione del problema.

Somma e differenza I polinomi somma e differenza sono definiti operando componente per componente sui rispettivi coefficienti dello stesso grado:

- $P_A + P_B = \sum_{i=0}^{\max(n,m)} (a_i + b_i) x^i$
- $P_A - P_B = \sum_{i=0}^{\max(n,m)} (a_i - b_i) x^i$

(assumendo $a_i = 0$ per $i > n$ e $b_i = 0$ per $i > m$).

Il grado del polinomio risultante soddisfa la disuguaglianza: $\deg(P_A \pm P_B) \leq \max(\deg(P_A), \deg(P_B))$. La disuguaglianza è stretta nel caso in cui si verifichi la cancellazione del termine di grado massimo.

Valgono inoltre le seguenti proprietà legate all'elemento neutro additivo (il polinomio nullo 0), fondamentali per la gestione dei casi base:

- $P_A + 0 = 0 + P_A = P_A$
- $P_A - 0 = P_A$
- $0 - P_A = -P_A$

Prodotto Siano $P_A, P_B \neq 0$. Il polinomio prodotto $P_C = P_A \cdot P_B$ ha grado $\deg(P_C) = \deg P_A + \deg P_B$. Il coefficiente c_k di grado k del polinomio risultato è determinato tramite la convoluzione discreta (ovvero la sommatoria dei prodotti degli elementi le cui posizioni sommano all'indice del coefficiente risultante) dei coefficienti dei polinomi operandi:

$$c_k = \sum_{i+j=k} a_i \cdot b_j = \sum_{i=0}^k a_i \cdot b_{k-i}$$

Nel caso in cui $P_A = 0$ oppure $P_B = 0$, il prodotto restituisce direttamente il polinomio nullo 0.

Divisione euclidea Dati due polinomi $P_A, P_B \in \mathbb{R}[x]$ con $P_B \neq 0$, esistono e sono unici due polinomi $Q, R \in \mathbb{R}[x]$ (rispettivamente quoziente e resto) tali che:

$$P_A = P_B \cdot Q + R, \quad \text{con } R = 0 \quad \text{oppure} \quad \deg(R) < \deg(P_B)$$

Massimo comune divisore Dati due polinomi $A, B \in \mathbb{R}[x]$, un polinomio D è un loro divisore comune se divide sia A sia B , cioè se esistono $C_A, C_B \in \mathbb{R}[x]$ tali che

$$A = D \cdot C_A \quad \text{e} \quad B = D \cdot C_B.$$

Un massimo comune divisore di A e B è un divisore comune D tale che ogni altro divisore comune di A e B divide D .

Nell'anello $\mathbb{R}[x]$ il massimo comune divisore è determinato a meno della moltiplicazione per una costante reale non nulla. Per individuarne un rappresentante unico si sceglie convenzionalmente il rappresentante monico. Dati $A, B \in \mathbb{R}[x]$ con $B \neq 0$, indicando con R il resto della divisione euclidea di A per B , vale la relazione

$$\text{MCD}(A, B) = \text{MCD}(B, R), \quad \text{dove } A = B \cdot Q + R.$$

Nel caso in cui il secondo polinomio sia nullo, si ha

$$\text{MCD}(A, 0) = \begin{cases} \text{monico}(A) & \text{se } A \neq 0, \\ 0 & \text{se } A = 0. \end{cases}$$

3 Progettazione dell'Algoritmo

3.1 Scelte di Progetto

I polinomi si prestano naturalmente a essere rappresentati mediante strutture dati lineari: si utilizza una lista di coefficienti in ordine crescente di grado, dove l'elemento di indice i corrisponde al coefficiente di x^i . La lista ha lunghezza variabile, poiché il grado dei polinomi in ingresso non è noto a priori.

I coefficienti sono di tipo reale approssimato in virgola mobile a doppia precisione. Si adotta una costante di tolleranza $\varepsilon = 10^{-6}$ per trattare i confronti con lo zero in modo robusto rispetto agli errori di arrotondamento.

Rappresentazione dei casi particolari Il polinomio nullo è rappresentato convenzionalmente dalla lista vuota, anziché da una lista contenente un singolo coefficiente pari a zero, questa scelta semplifica uniformemente i casi base di tutte le operazioni definite sui polinomi, poiché il polinomio nullo può essere riconosciuto verificando che la lista dei coefficienti sia vuota. Coerentemente, il grado del polinomio nullo, matematicamente indefinito, viene fissato per convenzione a 0, in modo da restituire sempre un valore intero ben definito in fase di stampa.

La costante di tolleranza ε non viene impiegata solamente per i confronti con lo zero, ma anche per riconoscere un coefficiente numericamente prossimo a 1. Ciò consente, in fase di formattazione dell'output, di omettere il coefficiente esplicito nei monomi di grado 1 o superiore.

Va infine osservato un compromesso legato all'ordine crescente di grado scelto per la rappresentazione. Poiché il coefficiente direttore di un polinomio corrisponde sempre all'ultimo elemento della lista, la sua estrazione (necessaria ad ogni passo della divisione lunga e nella resa monica dell'MCD) richiede un attraversamento completo della lista, con costo $O(n)$. Si è comunque preferita questa rappresentazione, che garantisce una corrispondenza diretta e immediata tra indice e grado, rispetto a una rappresentazione a grado decrescente (che avrebbe reso l'accesso al coefficiente direttore $O(1)$, ma avrebbe reso meno immediate la costruzione e l'interpretazione delle operazioni).

Formato di acquisizione dei coefficienti Ciascun polinomio viene acquisito digitando su un'unica riga i suoi coefficienti, separati da uno o più spazi o tabulazioni ed elencati in ordine crescente di grado. Il separatore decimale è il punto ed è ammessa la notazione scientifica (ad esempio $1\text{e-}3$ oppure $1.5\text{E}2$), non sono invece ammesse le notazioni esadecimale e ottale, così da garantire un formato di ingresso identico nei due programmi. È richiesto almeno un coefficiente esplicito, il polinomio nullo si ottiene digitando 0.

Formattazione dell'output I polinomi vengono stampati in forma algebrica canonica, dal grado massimo al grado minimo, omettendo i termini nulli, i coefficienti unitari e le potenze x^0 e x^1 . Ogni coefficiente è stampato come intero quando la sua parte decimale è trascurabile rispetto a ε ; in caso contrario viene arrotondato a quattro posizioni decimali, omettendo gli zeri finali. I valori il cui modulo supera 10^{14} non vengono arrotondati a intero, per evitare una perdita significativa di informazione.

Normalizzazione numerica A causa della natura dell'aritmetica in virgola mobile, coefficienti teoricamente nulli possono assumere piccoli valori spuri. Sfruttando la già citata soglia di tolleranza, ogni coefficiente c_i tale che $|c_i| < \varepsilon$ viene pertanto trattato come zero.

La *normalizzazione numerica* consiste nell'eliminazione di tutti questi coefficienti nulli di grado massimo. Questa operazione garantisce che il coefficiente direttore del polinomio finale sia significativamente diverso da zero, oppure, nel caso del polinomio nullo, che la lista dei coefficienti risulti vuota. Tale accorgimento è indispensabile affinché il grado del polinomio sia determinato in modo robusto e coerente durante l'esecuzione algebrica.

La normalizzazione viene applicata dopo l'acquisizione, nel calcolo del grado, nella divisione e nell'MCD, nonché durante la formattazione dell'output. Non viene invece applicata sistematicamente al risultato di somma, differenza e prodotto, poiché le operazioni che dipendono dal grado normalizzano comunque i propri argomenti prima di utilizzarli.

Limitazioni Le scelte di progetto descritte comportano le seguenti limitazioni:

- La tolleranza ε è assoluta e non relativa: un polinomio i cui coefficienti siano tutti di modulo inferiore a 10^{-6} viene quindi riconosciuto come il polinomio nullo. Digitando, ad esempio, 0.0000001 si ottiene il polinomio 0.
- I coefficienti sono reali approssimati in virgola mobile a doppia precisione, per cui i risultati sono soggetti agli errori di arrotondamento tipici di tale aritmetica. In particolare, l'MCD di polinomi aventi un fattore comune viene riconosciuto solo se la cancellazione dei coefficienti avviene entro la soglia di tolleranza.
- La validazione dell'input impedisce l'acquisizione diretta di coefficienti non finiti. Tuttavia, operazioni aritmetiche eseguite su coefficienti finiti di modulo molto elevato possono produrre risultati intermedi non finiti a causa di overflow. L'implementazione corrente non verifica la finitezza di tali risultati intermedi; pertanto, in questi casi, non sono garantite né la correttezza dei risultati né la terminazione della divisione e del calcolo dell'MCD. Si assume quindi che i coefficienti acquisiti abbiano valori tali da mantenere tutti i risultati intermedi entro l'intervallo rappresentabile in virgola mobile a doppia precisione.

3.2 Passi dell'Algoritmo

I passi dell'algoritmo per risolvere il problema sono i seguenti.

- **Acquisire il primo polinomio P_A :** leggere la sequenza di coefficienti reali fornita dall'utente e normalizzare la lista risultante. Se la riga inserita è vuota, oppure contiene un token che non rappresenta un numero reale finito (perché malformato o perché affetto da overflow), segnalare l'errore all'utente e ripetere l'acquisizione.
- **Acquisire il secondo polinomio P_B :** analogo al passo precedente.
- **Stampare i polinomi acquisiti:** mostrare P_A e P_B nella loro forma algebrica canonica.
- **Calcolare e stampare il grado di P_A e di P_B :** il grado di un polinomio è pari all'indice del coefficiente non nullo di grado massimo, mentre per il polinomio nullo si restituisce il valore convenzionale 0.
- **Calcolare e stampare la somma $P_A + P_B$:**
 - Caso base: se uno dei due polinomi è il polinomio nullo, la somma è uguale all'altro polinomio.
 - Caso generale: sommare termine a termine i coefficienti del medesimo grado, poi procedere ricorsivamente con i coefficienti restanti.
- **Calcolare e stampare la differenza $P_A - P_B$:**
 - Caso base (primo polinomio nullo): il risultato è l'opposto dei coefficienti rimanenti del secondo polinomio.
 - Caso base (secondo polinomio nullo): il risultato è il primo polinomio invariato.
 - Caso generale: sottrarre termine a termine, poi procedere ricorsivamente.

- **Calcolare e stampare il prodotto $P_A \cdot P_B$:**
 - Caso base: se uno dei due polinomi è il polinomio nullo, il prodotto è il polinomio nullo.
 - Caso generale: moltiplicare il primo coefficiente di P_A per tutti i coefficienti di P_B , poi sommare al risultato il prodotto tra il resto di P_A e P_B riscalato di un grado (equivalente alla moltiplicazione per x , ovvero all’inserimento di uno zero in testa alla lista).
- **Calcolare e stampare quoziente e resto della divisione euclidea $P_A \div P_B$:**
 - Se il divisore P_B è il polinomio nullo, interrompere il calcolo e segnalare all’utente l’impossibilità di eseguire la divisione.
 - Normalizzare dividendo e divisore.
 - Caso base: se il grado del dividendo corrente è inferiore al grado del divisore, il quoziente accumulato è il quoziente finale e il dividendo corrente è il resto.
 - Caso generale: estrarre il coefficiente direttore del dividendo e quello del divisore, calcolare il coefficiente del monomio parziale del quoziente come loro rapporto, moltiplicare il divisore per tale monomio e sottrarlo dal dividendo corrente, accumulare il monomio al quoziente, ripetere ricorsivamente sul dividendo ridotto.
- **Calcolare e stampare il massimo comune divisore $\text{MCD}(P_A, P_B)$:**
 - Caso base: se il secondo polinomio è il polinomio nullo, il risultato è il primo polinomio reso monico.
 - Caso generale: calcolare il resto R della divisione euclidea di P_A per P_B e ripetere ricorsivamente con la coppia (P_B, R) .

4 Implementazione dell'Algoritmo

4.1 Differenze Implementative

Le differenze implementative tra le due versioni sono le seguenti.

- **Sistema dei tipi e validazione dell'input:** in Haskell il tipo dei coefficienti è `Double` e la correttezza dei tipi è controllata staticamente dal compilatore. La funzione `leggi_coefficiente`, richiamata dall'azione `acquisisci_polinomio`, controlla anzitutto che il token digitato contenga solo caratteri ammessi in un coefficiente reale (cifre, punto, segno ed esponente), poi utilizza `readMaybe` del modulo `Text.Read` per eseguire la conversione senza generare eccezioni. Il controllo preliminare esclude esplicitamente le sintassi non ammesse, tra cui le notazioni esadecimale e ottale di Haskell, estranee al programma Prolog. La funzione `verifica_coefficiente` scarta infine i valori non finiti prodotti dall'overflow della conversione. Se una qualsiasi di queste verifiche fallisce, viene segnalato un errore e l'acquisizione viene ripetuta. In Prolog non esiste un sistema di tipi statico, ma si è ottenuto un comportamento robusto limitatamente alla fase di acquisizione: gli errori prodotti dalla conversione dei token vengono intercettati e, in caso di formato non valido, l'acquisizione viene ripetuta in modo controllato. Poiché lo standard ISO non prevede alcun predicato predefinito per la lettura di una riga di testo, il predicato `leggi_riga/1` la acquisisce carattere per carattere mediante `get_char/1`, riconoscendo sia il fine riga sia la fine dell'input (a differenza di `read/1`, questa lettura non richiede quindi che l'input termini con il punto, essendo priva di vincoli sintattici legati ai termini Prolog), il predicato `dividi_in_token/2` suddivide quindi la riga sugli spazi e sulle tabulazioni, e il predicato `converti_input/2`, richiamato da `elabora_input/3`, tenta la conversione sicura di ciascun token con `catch(number_chars(N, T), _, fail)`, verificandone poi la finitezza con `coefficiente_valido/1`. Poiché `number_chars/2` distingue interi e reali, il predicato `aggiungi_punto_decimale/2` porta preventivamente ogni token nella forma sintattica di un numero reale, inserendo la parte decimale nulla prima dell'eventuale esponente (riconosciuto da `esponente/1`) oppure, in sua assenza, in coda al token: in questo modo tutti i coefficienti sono uniformemente reali, come i `Double` di Haskell. In caso di errore, il predicato fallisce controllatamente, avvisa l'utente e reitera l'acquisizione.

- **Stile dichiarativo e ricorsione:** in Haskell le funzioni sono definite per pattern matching esaustivo sulle liste e composizione funzionale. Non vengono utilizzati costrutti imperativi (**if-then-else**): la condizione di guardia (`| ... = ...`) sostituisce il costrutto condizionale sia nella formattazione dell'output, sia nelle funzioni di calcolo come **divisione**, sia nella gestione dell'errore di lettura (**gestisci_errore_lettura**). In Prolog le medesime funzionalità sono espresse tramite clausole multiple mutuamente esclusive. Il taglio (!) è utilizzato per prevenire backtracking indesiderato non solo nei predicati di normalizzazione (**normalizza_lista_invertita/2**) e di formattazione, ma anche per rendere deterministici i casi base delle operazioni aritmetiche.
- **Divisione euclidea:** in Haskell il quoziente viene accumulato dalla funzione di supporto **divisione_ricorsiva**, che utilizza un accumulatore per il quoziente (parametro **quoziente**, aggiornato ad ogni passo tramite il binding locale **quoziente_aggiornato**) e restituisce la coppia (**quoziente**, **resto**) al termine della ricorsione, la scelta tra caso base e caso generale è espressa da una guardia sulle lunghezze delle due liste. In Prolog l'equivalente predicato **divisione_ricorsiva/5** gestisce esplicitamente cinque argomenti. Poiché in Prolog la distinzione tra i due casi non può essere espressa da una guardia, il predicato calcola una sola volta le lunghezze delle due liste e le passa a **passo_divisione/7**, le cui due clausole realizzano rispettivamente il caso base e il caso generale.
- **Segnalazione dell'impossibilità di dividere:** in Haskell la funzione **divisione** restituisce un valore di tipo **Maybe** (`[Double], [Double]`), pari a **Nothing** quando il divisore è il polinomio nullo, l'azione **main** distingue i due casi con un **case** sul risultato. In Prolog il predicato **divisione/4** fallisce nello stesso caso, e la distinzione è affidata alle due clausole del predicato **gestisci_divisione/2**, che non ha un corrispondente in Haskell.
- **Produzione dell'output:** in Haskell le funzioni di formattazione sono pure e restituiscono la stringa da stampare (**mostra**, con le ausiliarie **termini**, **segno**, **monomio** e **formatta_coefficiente**), che l'azione **main** affida a **putStrLn**. In Prolog i predicati corrispondenti stampano direttamente tramite **write/1**, da cui i nomi **formatta_termini/2**, **formatta_segno/2**, **formatta_monomio/2** e **scrivi_valore/1**.

- **Funzioni di libreria:** in Haskell la moltiplicazione di un polinomio per uno scalare e il cambiamento di segno sono ottenuti applicando `map` alle sezioni (`* testa_a`) e `negate`, mentre la normalizzazione sfrutta `dropWhileEnd` del modulo `Data.List`. Nell'implementazione Prolog, senza ricorrere a predicati di ordine superiore, le stesse operazioni sono realizzate dal predicato ricorsivo `moltiplica_per_scalare/3` e dal predicato `normalizza/2`, che inverte la lista, elimina i coefficienti nulli in testa e la inverte nuovamente.

4.2 File sorgente Haskell (operazioni_polinomi.hs)

```
{- Programma Haskell per operazioni ed algoritmi su polinomi. -}

module Main where

-- necessario per disabilitare il buffering dell'output
import System.IO (hSetBuffering, stdout, BufferMode (..))
-- necessario per il parsing sicuro dei coefficienti da tastiera
import Text.Read (readMaybe)
-- necessario per riconoscere le cifre nei token digitati dall'utente
import Data.Char (isDigit)
-- necessario per rimuovere i coefficienti nulli di grado massimo
import Data.List (dropWhileEnd)
{- necessario per intercettare l'EOF da tastiera in modo pulito
    (invece di lasciar propagare l'eccezione non gestita di getLine) -}
import System.IO.Error (isEOFError, catchIOError)
-- necessario per terminare pulitamente il programma in caso di EOF
import System.Exit (exitSuccess)

{- La costante tolleranza rappresenta la soglia al di sotto della
    quale un valore Double viene considerato pari a zero, al fine di
    compensare gli errori di arrotondamento tipici dell'aritmetica in
    virgola mobile -}

tolleranza :: Double
tolleranza = 0.000001

{- L'azione main coordina l'intero programma: acquisisce i due
    polinomi, li visualizza, calcola e stampa il grado di ciascuno, la
    loro somma, differenza e prodotto, il quoziente e il resto della
    divisione euclidea (segnalando l'eventuale impossibilità di
    dividere per il polinomio nullo) e il loro massimo comune divisore -}

main :: IO ()
main = do
    hSetBuffering stdout NoBuffering
    polinomio_a <- acquisisci_polinomio "A"
    polinomio_b <- acquisisci_polinomio "B"
    putStrLn "\n"
    putStrLn $ "Polinomio A:      " ++ mostra polinomio_a
    putStrLn $ "Polinomio B:      " ++ mostra polinomio_b
    putStrLn $ "Grado A:          " ++ show (grado_polinomio polinomio_a)
    putStrLn $ "Grado B:          " ++ show (grado_polinomio polinomio_b)
    putStrLn $ "Somma:            " ++
        mostra (somma polinomio_a polinomio_b)
    putStrLn $ "Differenza:       " ++
        mostra (differenza polinomio_a polinomio_b)
    putStrLn $ "Prodotto:         " ++
        mostra (prodotto polinomio_a polinomio_b)
    case divisione polinomio_a polinomio_b of
        Nothing -> putStrLn ("Errore:            impossibile dividere " ++
            "per il polinomio nullo.")
        Just (quoziente, resto) -> do
            putStrLn $ "Quoziente:        " ++ mostra quoziente
            putStrLn $ "Resto:            " ++ mostra resto
            putStrLn $ "MCD:              " ++
```



```

        mostra (mcd polinomio_a polinomio_b)

{- L'azione parametrica di input/output acquisisci_polinomio
   acquisisce un polinomio di coefficienti Double da tastiera,
   restituendo la lista dei coefficienti in ordine crescente di grado:
   - il suo unico argomento è l'etichetta (nome) del polinomio da
     acquisire, usata nel messaggio mostrato all'utente
   Se l'utente chiude lo standard input (EOF) invece di digitare una
   riga, il programma termina con un messaggio invece di sollevare
   un'eccezione non gestita -}

acquisisci_polinomio :: String -> IO [Double]
acquisisci_polinomio etichetta = do
    putStr $ "Inserisci i coefficienti del polinomio " ++
        etichetta ++
        " separati da spazi (ordine crescente di grado): "
    riga_letta <- leggi_riga_sicura
    case riga_letta of
        Nothing -> do
            putStrLn "\nInput terminato (EOF). Uscita dal programma."
            exitSuccess
        Just riga -> case words riga of
            [] -> putStrLn "Devi inserire almeno un coefficiente esplicito!"
                >> acquisisci_polinomio etichetta
            token_letti -> case mapM leggi_coefficiente token_letti of
                Just coeff -> return (normalizza coeff)
                Nothing -> putStrLn "Formato non valido! Riprova." >>
                    acquisisci_polinomio etichetta

{- L'azione leggi_riga_sicura legge una riga da tastiera restituendo
   Nothing se lo standard input è terminato (EOF), invece di lasciar
   propagare l'eccezione di getLine -}

leggi_riga_sicura :: IO (Maybe String)
leggi_riga_sicura =
    (Just <$> getLine) `catchIOError` gestisci_errore_lettura
  where
    {- L'azione ausiliaria gestisci_errore_lettura restituisce Nothing
       se l'errore intercettato segnala la fine dell'input, altrimenti
       propaga l'errore:
       - il suo unico argomento è l'errore di input/output
         intercettato -}

    gestisci_errore_lettura :: IOError -> IO (Maybe String)
    gestisci_errore_lettura errore
        | isEOFError errore = return Nothing
        | otherwise         = ioError errore

{- La funzione leggi_coefficiente converte un token digitato
   dall'utente nel corrispondente coefficiente, restituendo Nothing se
   il token non rappresenta un numero reale finito in notazione
   decimale o scientifica:
   - il suo unico argomento è il token da convertire
   Il controllo preliminare sui caratteri ammessi esclude le notazioni
   esadecimale e ottale di Haskell (0x10, 0o10), che altrimenti
   sarebbero accettate da readMaybe ma non dal programma Prolog -}

```

```

leggi_coefficiente :: String -> Maybe Double
leggi_coefficiente token
  | all carattere_ammesso token = verifica_coefficiente (readMaybe token)
  | otherwise                    = Nothing
where
  {- La funzione ausiliaria carattere_ammesso riconosce i caratteri
     che possono comparire in un coefficiente reale:
     - il suo unico argomento è il carattere da riconoscere -}

  carattere_ammesso :: Char -> Bool
  carattere_ammesso carattere = isDigit carattere ||
                                elem carattere "+-eE"

{- La funzione verifica_coefficiente scarta i valori non finiti
   prodotti dall'overflow della conversione (ad esempio 1e400), che
   renderebbero non terminante il calcolo della divisione e del
   massimo comune divisore:
   - il suo unico argomento è l'esito della conversione del token -}

verifica_coefficiente :: Maybe Double -> Maybe Double
verifica_coefficiente Nothing = Nothing
verifica_coefficiente (Just coefficiente)
  | isNaN coefficiente || isInfinite coefficiente = Nothing
  | otherwise                                     = Just coefficiente

{- La funzione normalizza elimina i coefficienti nulli di grado
   massimo di un polinomio, ossia quelli che si trovano in coda alla
   lista:
   - il suo unico argomento è la lista dei coefficienti in ordine
     crescente di grado -}

normalizza :: [Double] -> [Double]
normalizza = dropWhileEnd (\c -> abs c < tolleranza)

{- La funzione mostra restituisce la rappresentazione algebrica di un
   polinomio, dal grado massimo al minimo:
   - il suo unico argomento è la lista dei coefficienti in ordine
     crescente di grado

   Le funzioni ausiliarie termini, segno e monomio dipendono dalla
   struttura del polinomio da stampare e sono usate solo qui, da cui
   il where; le funzioni di formattazione del singolo coefficiente
   sono invece indipendenti dal polinomio e definite a livello
   globale -}

mostra :: [Double] -> String
mostra polinomio =
  termini (reverse (zip [0..] (normalizza polinomio))) True
where
  {- La funzione ausiliaria termini concatena ricorsivamente i
     termini del polinomio:
     - il primo argomento è la lista delle coppie (grado,
       coefficiente) residue da stampare, ordinata dal grado
       massimo al minimo
     - il secondo argomento è il flag che indica se il termine da
       stampare è il primo termine non nullo del polinomio -}

  termini :: [(Int, Double)] -> Bool -> String

```

```

termini [] True  = "0"
termini [] False = ""
termini ((grado, c) : resto) primo
  | abs c < tolleranza = termini resto primo
  | otherwise          = segno c primo ++
                        monomio grado (abs c) ++
                        termini resto False

{- La funzione ausiliaria segno restituisce il segno da anteporre
   al termine corrente:
   - il primo argomento è il coefficiente del termine corrente
   - il secondo argomento è il flag che indica se il termine è il
     primo termine non nullo del polinomio -}

segno :: Double -> Bool -> String
segno c True  | c < 0      = "-"
              | otherwise = ""
segno c False | c < 0      = " - "
              | otherwise = " + "

{- La funzione ausiliaria monomio formatta grado e coefficiente,
   omettendo i coefficienti unitari e le potenze 0 e 1:
   - il primo argomento è il grado del monomio
   - il secondo argomento è il valore assoluto del coefficiente
     del monomio -}

monomio :: Int -> Double -> String
monomio 0 c = formatta_coefficiente c
monomio 1 c | abs (c - 1) < tolleranza = "x"
            | otherwise = formatta_coefficiente c ++ "x"
monomio g c | abs (c - 1) < tolleranza = "x^" ++ show g
            | otherwise = formatta_coefficiente c ++
                          "x^" ++ show g

{- La funzione formatta_coefficiente restituisce la rappresentazione
   testuale di un coefficiente non negativo, come intero se la parte
   decimale è trascurabile, altrimenti arrotondato a quattro cifre
   decimali:
   - il suo unico argomento è il valore del coefficiente
   Per valori il cui modulo supera 1e14 si evita la conversione a Int
   (che risulterebbe inesatta vicino ai limiti di Int) e si delega la
   stampa a show sul Double originale -}

formatta_coefficiente :: Double -> String
formatta_coefficiente x
  | abs x > 1.0e14 = show x
  | abs (x - fromIntegral (round x :: Int)) < tolleranza =
    show (round x :: Int)
  | otherwise = componi_parti parte_intera parte_frazionaria
where
  parte_intera      = truncate x :: Int
  frazione          = x - fromIntegral parte_intera
  parte_frazionaria = round (frazione * 10000) :: Int

{- La funzione componi_parti concatena la parte intera e, se
   significativa, la parte frazionaria di un coefficiente, gestendo
   il riporto prodotto dall'arrotondamento alla quarta cifra

```

```

decimale:
- il primo argomento è la parte intera del coefficiente
- il secondo argomento è la parte frazionaria arrotondata,
  espressa come intero su quattro cifre -}

componi_parti :: Int -> Int -> String
componi_parti parte_intera 10000 = show (parte_intera + 1)
componi_parti parte_intera 0     = show parte_intera
componi_parti parte_intera parte_frazionaria =
  show parte_intera ++ "." ++
  cifre_frazionarie parte_frazionaria 1000

{- La funzione cifre_frazionarie concatena le cifre della parte
   frazionaria, dalla più significativa alla meno significativa,
   arrestandosi quando le cifre residue sono tutte nulle:
- il primo argomento è la parte frazionaria residua, espressa come
  intero su quattro cifre
- il secondo argomento è il peso della cifra da concatenare -}

cifre_frazionarie :: Int -> Int -> String
cifre_frazionarie 0 _ = ""
cifre_frazionarie parte_frazionaria peso =
  show (parte_frazionaria `div` peso) ++
  cifre_frazionarie (parte_frazionaria `mod` peso) (peso `div` 10)

{- La funzione grado_polinomio calcola il grado di un polinomio come
   la lunghezza del polinomio normalizzato meno uno, oppure 0 se il
   polinomio è nullo:
- il suo unico argomento è la lista dei coefficienti in ordine
  crescente di grado -}

grado_polinomio :: [Double] -> Int
grado_polinomio coeff = case normalizza coeff of
  [] -> 0
  norm -> length norm - 1

{- La funzione somma calcola la somma di due polinomi in tempo
   lineare O(n):
- il primo argomento è la lista dei coefficienti del primo
  polinomio
- il secondo argomento è la lista dei coefficienti del secondo
  polinomio
il risultato è la lista dei coefficienti del polinomio somma, non
normalizzata -}

somma :: [Double] -> [Double] -> [Double]
somma [] coeff_b = coeff_b
somma coeff_a [] = coeff_a
somma (testa_a : resto_a) (testa_b : resto_b) =
  (testa_a + testa_b) : somma resto_a resto_b

{- La funzione differenza calcola la differenza tra due polinomi in
   tempo lineare O(n):
- il primo argomento è la lista dei coefficienti del primo
  polinomio
- il secondo argomento è la lista dei coefficienti del secondo
  polinomio

```

```
    il risultato è la lista dei coefficienti del polinomio differenza,
    non normalizzata -}

differenza :: [Double] -> [Double] -> [Double]
differenza [] coeff_b = map negate coeff_b
differenza coeff_a [] = coeff_a
differenza (testa_a : resto_a) (testa_b : resto_b) =
    (testa_a - testa_b) : differenza resto_a resto_b

{- La funzione prodotto calcola il prodotto di due polinomi:
    - il primo argomento è la lista dei coefficienti del primo
      polinomio
    - il secondo argomento è la lista dei coefficienti del secondo
      polinomio
    il risultato è la lista dei coefficienti del polinomio prodotto,
    non normalizzata -}

prodotto :: [Double] -> [Double] -> [Double]
prodotto [] _ = []
prodotto _ [] = []
prodotto (testa_a : resto_a) polinomio_b =
    somma (map (* testa_a) polinomio_b)
    (0 : prodotto resto_a polinomio_b)

{- La funzione divisione calcola quoziente e resto della divisione
    euclidea tra due polinomi:
    - il primo argomento è la lista dei coefficienti del polinomio
      dividendo
    - il secondo argomento è la lista dei coefficienti del polinomio
      divisore
    il risultato, se il divisore non è il polinomio nullo, è la coppia
    formata dalla lista dei coefficienti del polinomio quoziente e
    dalla lista dei coefficienti del polinomio resto -}

divisione :: [Double] -> [Double] -> Maybe ([Double], [Double])
divisione dividendo divisore
    | null (normalizza divisore) = Nothing
    | otherwise = Just (divisione_ricorsiva (normalizza dividendo)
                                             (normalizza divisore)
                                             [])

where
    {- La funzione ausiliaria divisione_ricorsiva esegue la divisione
        lunga accumulando il quoziente:
        - il primo argomento è la lista dei coefficienti del dividendo
          corrente, normalizzata
        - il secondo argomento è la lista dei coefficienti del
          divisore, normalizzata
        - il terzo argomento è la lista dei coefficienti del quoziente
          parziale accumulato finora -}

    divisione_ricorsiva :: [Double] -> [Double] -> [Double] ->
        ([Double], [Double])
    divisione_ricorsiva dividendo_corrente divisore_norm quoziente
        | length dividendo_corrente < length divisore_norm =
            (normalizza quoziente, normalizza dividendo_corrente)
        | otherwise = divisione_ricorsiva dividendo_aggiornato
            divisore_norm
```

```

                                quoziente_aggiornato
where
  coefficiente_termine = last dividendo_corrente /
                        last divisore_norm
  termine_quoziente   = replicate (length dividendo_corrente -
                                length divisore_norm) 0 ++
                        [coefficiente_termine]
  dividendo_aggiornato = normalizza
                        (differenza dividendo_corrente
                        (prodotto divisore_norm
                        termine_quoziente))
  quoziente_aggiornato = somma quoziente termine_quoziente

{- La funzione mcd calcola il massimo comune divisore di due polinomi
tramite l'algoritmo di Euclide, restituendo il risultato reso
monico:
- il primo argomento è la lista dei coefficienti del primo
  polinomio
- il secondo argomento è la lista dei coefficienti del secondo
  polinomio
il risultato è la lista dei coefficienti del polinomio massimo
comun divisore -}

mcd :: [Double] -> [Double] -> [Double]
mcd polinomio_a polinomio_b = euclide (normalizza polinomio_a)
                                     (normalizza polinomio_b)

where
  {- La funzione ausiliaria euclide applica ricorsivamente
l'algoritmo di Euclide ai due polinomi, tramite monico, fino a
ridurre il secondo polinomio al polinomio nullo:
- il primo argomento è la lista dei coefficienti del primo
  polinomio, normalizzata
- il secondo argomento è la lista dei coefficienti del secondo
  polinomio, normalizzata
Caso limite: se entrambi i polinomi sono nulli, il risultato è
il polinomio nullo -}

euclide :: [Double] -> [Double] -> [Double]
euclide primo [] = monico primo
euclide primo secondo = euclide secondo
                      (normalizza (resto_divisione primo
                      secondo))

{- La funzione ausiliaria monico divide tutti i coefficienti di
un polinomio per il suo coefficiente direttore (l'ultimo della
lista):
- il suo unico argomento è la lista dei coefficienti da
  rendere monica -}

monico :: [Double] -> [Double]
monico [] = []
monico coeff = map (/ last coeff) coeff

{- La funzione ausiliaria resto_divisione estrae il resto della
divisione euclidea tra due polinomi:
- il primo argomento è la lista dei coefficienti del dividendo
- il secondo argomento è la lista dei coefficienti del divisore

```

Il divisore non è mai il polinomio nullo quando la funzione
viene invocata da euclide, per cui il secondo caso non si
verifica mai ed è presente solo per esaustività -}

```
resto_divisione :: [Double] -> [Double] -> [Double]
resto_divisione primo secondo = case divisione primo secondo of
  Just (_, resto) -> resto
  Nothing          -> []
```

4.3 File sorgente Prolog (operazioni_polinomi.pl)

```

/* Programma Prolog per operazioni ed algoritmi su polinomi. */

/* La costante tolleranza rappresenta la soglia al di sotto della
   quale un valore reale viene considerato pari a zero, al fine di
   compensare gli errori di arrotondamento tipici dell'aritmetica in
   virgola mobile */

tolleranza_numerica(0.000001).

/* Il predicato main coordina l'intero programma: acquisisce i due
   polinomi, li visualizza, calcola e stampa il grado di ciascuno, la
   loro somma, differenza e prodotto, il quoziente e il resto della
   divisione euclidea (segnalando l'eventuale impossibilità di
   dividere per il polinomio nullo) e il loro massimo comune divisore */

main :-
    acquisisci_polinomio('A', Primo_poli),
    acquisisci_polinomio('B', Secondo_poli),
    nl, nl,
    write('Polinomio A:      '), mostra(Primo_poli), nl,
    write('Polinomio B:      '), mostra(Secondo_poli), nl,
    grado_polinomio(Primo_poli, Grado_a),
    write('Grado A:          '), write(Grado_a), nl,
    grado_polinomio(Secondo_poli, Grado_b),
    write('Grado B:          '), write(Grado_b), nl,
    somma(Primo_poli, Secondo_poli, Somma),
    write('Somma:           '), mostra(Somma), nl,
    differenza(Primo_poli, Secondo_poli, Differenza),
    write('Differenza:        '), mostra(Differenza), nl,
    prodotto(Primo_poli, Secondo_poli, Prodotto),
    write('Prodotto:          '), mostra(Prodotto), nl,
    gestisci_divisione(Primo_poli, Secondo_poli),
    mcd(Primo_poli, Secondo_poli, Mcd),
    write('MCD:              '), mostra(Mcd), nl.

/* Il predicato acquisisci_polinomio acquisisce un polinomio a
   coefficienti reali da tastiera, restituendo la lista dei
   coefficienti in ordine crescente di grado:
   - il primo argomento è l'etichetta (nome) del polinomio da
     acquisire, usata nel messaggio mostrato all'utente
   - il secondo argomento è la lista dei coefficienti acquisita */

acquisisci_polinomio(Etichetta, Polinomio) :-
    write('Inserisci i coefficienti del polinomio '), write(Etichetta),
    write(' separati da spazi (ordine crescente di grado): '),
    leggi_riga(Caratteri),
    interpreta_riga(Etichetta, Caratteri, Polinomio).

/* Il predicato leggi_riga legge da tastiera una riga di testo,
   carattere per carattere, fino al primo fine riga oppure alla fine
   dell'input:
   - il suo unico argomento è la lista dei caratteri letti, oppure
     l'atomo fine_input se l'input è terminato */

leggi_riga(Riga) :-

```



```
    get_char(Carattere),
    apri_riga(Carattere, Riga).

/* Il predicato ausiliario apri_riga distingue la fine dell'input dal
primo carattere di una riga effettiva:
- il primo argomento è il primo carattere letto
- il secondo argomento è la riga risultante, ossia la lista dei
  suoi caratteri oppure l'atomo fine_input */

apri_riga(end_of_file, fine_input) :- !.
apri_riga(Carattere, Riga) :-
    accumula_caratteri(Carattere, Riga).

/* Il predicato ausiliario accumula_caratteri costruisce
ricorsivamente la lista dei caratteri della riga corrente,
ignorando i ritorni carrello dei file di testo in formato Windows:
- il primo argomento è il carattere corrente
- il secondo argomento è la lista dei caratteri risultante */

accumula_caratteri(end_of_file, []) :- !.
accumula_caratteri('\n', []) :- !.
accumula_caratteri('\r', Caratteri) :- !,
    get_char(Successivo),
    accumula_caratteri(Successivo, Caratteri).
accumula_caratteri(Carattere, [Carattere|Resto]) :-
    get_char(Successivo),
    accumula_caratteri(Successivo, Resto).

/* Il predicato ausiliario interpreta_riga termina il programma se
l'input è esaurito, altrimenti suddivide la riga letta nei token
che la compongono:
- il primo argomento è l'etichetta del polinomio, usata per
  richiedere nuovamente l'input in caso di errore
- il secondo argomento è la riga letta, ossia la lista dei suoi
  caratteri oppure l'atomo fine_input
- il terzo argomento è la lista dei coefficienti risultante */

interpreta_riga(_, fine_input, _) :- !,
    nl, write('Input terminato (EOF). Uscita dal programma.'), nl,
    halt.
interpreta_riga(Etichetta, Caratteri, Polinomio) :-
    dividi_in_token(Caratteri, Token),
    prosegui_acquisizione(Etichetta, Token, Polinomio).

/* Il predicato dividi_in_token spezza una lista di caratteri sugli
spazi e sulle tabulazioni, restituendo la lista dei token che la
compongono (ciascuno a sua volta una lista di caratteri) e
scartando i token vuoti generati da separatori consecutivi:
- il primo argomento è la lista dei caratteri da suddividere
- il secondo argomento è la lista dei token risultante */

dividi_in_token(Caratteri, Token) :-
    dividi_in_token(Caratteri, [], [], Token).

/* Il predicato ausiliario dividi_in_token/4 scandisce ricorsivamente
la lista dei caratteri, accumulando il token corrente e
chiudendolo ogni volta che incontra uno spazio o una tabulazione:
```

```

- il primo argomento è la lista dei caratteri residui da scandire
- il secondo argomento è l'accumulatore (invertito) del token
  corrente
- il terzo argomento è l'accumulatore (invertito) dei token già
  chiusi
- il quarto argomento è la lista dei token risultante, in ordine */

dividi_in_token([], Acc_corrente, Acc_token, Token) :-
  aggiungi_token(Acc_corrente, Acc_token, Token_invertiti),
  reverse(Token_invertiti, Token).
dividi_in_token([' '|Resto], Acc_corrente, Acc_token, Token) :- !,
  aggiungi_token(Acc_corrente, Acc_token, Acc_aggiornato),
  dividi_in_token(Resto, [], Acc_aggiornato, Token).
dividi_in_token(['\t'|Resto], Acc_corrente, Acc_token, Token) :- !,
  aggiungi_token(Acc_corrente, Acc_token, Acc_aggiornato),
  dividi_in_token(Resto, [], Acc_aggiornato, Token).
dividi_in_token([Carattere|Resto], Acc_corrente, Acc_token, Token) :-
  dividi_in_token(Resto, [Carattere|Acc_corrente], Acc_token, Token).

/* Il predicato ausiliario aggiungi_token chiude il token corrente e
   lo antepone alla lista dei token già accumulati, ignorando i token
   vuoti generati da separatori consecutivi:
- il primo argomento è il token corrente (invertito),
  eventualmente vuoto
- il secondo argomento è la lista (invertita) dei token già chiusi
- il terzo argomento è la lista (invertita) risultante */

aggiungi_token([], Acc_token, Acc_token) :- !.
aggiungi_token(Acc_corrente, Acc_token, [Token|Acc_token]) :-
  reverse(Acc_corrente, Token).

/* Il predicato ausiliario prosegui_acquisizione ripete l'acquisizione
   se la riga digitata non contiene alcun token, altrimenti avvia la
   conversione dei token in coefficienti:
- il primo argomento è l'etichetta del polinomio, usata per
  richiedere nuovamente l'input in caso di errore
- il secondo argomento è la lista dei token estratti dalla riga
- il terzo argomento è la lista dei coefficienti risultante */

prosegui_acquisizione(Etichetta, [], Polinomio) :- !,
  write('Devi inserire almeno un coefficiente esplicito!'), nl,
  acquisisci_polinomio(Etichetta, Polinomio).
prosegui_acquisizione(Etichetta, Token, Polinomio) :-
  elabora_input(Etichetta, Token, Polinomio).

/* Il predicato elabora_input converte i token in coefficienti
   numerici tramite converti_input, ripetendo l'acquisizione in caso
   di formato non valido:
- il primo argomento è l'etichetta del polinomio, usata per
  richiedere nuovamente l'input in caso di errore
- il secondo argomento è la lista dei token da convertire
- il terzo argomento è la lista dei coefficienti risultante */

elabora_input(_, Token, Polinomio) :-
  converti_input(Token, Coefficienti), !,
  normalizza(Coefficienti, Polinomio).
elabora_input(Etichetta, _, Polinomio) :-

```

```

write('Formato non valido! Riprova. '), nl,
acquisisci_polinomio(Etichetta, Polinomio).

/* Il predicato converti_input converte i token in coefficienti
   numerici, fallendo se anche un solo token non rappresenta un
   numero:
   - il primo argomento è la lista dei token da convertire
   - il secondo argomento è la lista dei coefficienti risultante */

converti_input(Token, Coefficienti) :-
    converti_input(Token, [], Coefficienti_invertiti),
    reverse(Coefficienti_invertiti, Coefficienti).

/* Il predicato ausiliario converti_input/3 converte ricorsivamente
   la lista dei token in coefficienti, accumulando il risultato in
   ordine inverso:
   - il primo argomento è la lista dei token residui da convertire
   - il secondo argomento è l'accumulatore dei coefficienti
     convertiti finora
   - il terzo argomento è la lista dei coefficienti risultante */

converti_input([], Acc, Acc).
converti_input([Token|Resto], Acc, Coefficienti) :-
    aggiungi_punto_decimale(Token, Token_reale),
    catch(number_chars(Numero, Token_reale), _, fail),
    Coefficiente is float(Numero),
    coefficiente_valido(Coefficiente),
    converti_input(Resto, [Coefficiente|Acc], Coefficienti).

/* Il predicato ausiliario aggiungi_punto_decimale porta un token
   numerico nella forma sintattica di un numero reale, inserendo la
   parte decimale nulla prima dell'eventuale esponente oppure, in sua
   assenza, in coda al token; un token che contiene già il punto
   decimale viene lasciato invariato:
   - il primo argomento è la lista dei caratteri del token originale
   - il secondo argomento è la lista dei caratteri risultante */

aggiungi_punto_decimale(Token, Token) :-
    member('.', Token), !.
aggiungi_punto_decimale(Token, Token_reale) :-
    append(Mantissa, [Carattere|Cifre], Token),
    esponente(Carattere), !,
    append(Mantissa, ['.', '0', Carattere|Cifre], Token_reale).
aggiungi_punto_decimale(Token, Token_reale) :-
    append(Token, ['.', '0'], Token_reale).

/* Il predicato ausiliario esponente riconosce i caratteri che
   introducono l'esponente nella notazione scientifica:
   - il suo unico argomento è il carattere da riconoscere */

esponente('e').
esponente('E').

/* Il predicato ausiliario coefficiente_valido scarta i valori non
   finiti prodotti dall'overflow della conversione (ad esempio 1e400),
   che renderebbero non terminante il calcolo della divisione e del
   massimo comune divisore:

```

```
- il suo unico argomento è il coefficiente da controllare */

coefficiente_valido(Coefficiente) :-
    abs(Coefficiente) =< 1.0e308.

/* Il predicato normalizza elimina i coefficienti nulli di grado
   massimo di un polinomio, ossia quelli che si trovano in coda alla
   lista:
   - il primo argomento è la lista dei coefficienti in ordine
     crescente di grado
   - il secondo argomento è la lista dei coefficienti risultante */

normalizza(Polinomio, Polinomio_norm) :-
    reverse(Polinomio, Invertito),
    normalizza_lista_invertita(Invertito, Invertito_ripulito),
    reverse(Invertito_ripulito, Polinomio_norm).

/* Il predicato ausiliario normalizza_lista_invertita elimina
   ricorsivamente i coefficienti di testa della lista invertita il cui
   valore assoluto è sotto tolleranza:
   - il primo argomento è la lista dei coefficienti in ordine
     invertito (dal grado massimo al minimo)
   - il secondo argomento è la lista risultante, priva dei
     coefficienti di testa sotto tolleranza */

normalizza_lista_invertita([Coefficiente|Resto], Risultato) :-
    tolleranza_numerica(T), abs(Coefficiente) < T, !,
    normalizza_lista_invertita(Resto, Risultato).
normalizza_lista_invertita(Lista, Lista).

/* Il predicato mostra stampa la rappresentazione algebrica di un
   polinomio, dal grado massimo al minimo:
   - il suo unico argomento è la lista dei coefficienti del polinomio
     da stampare, in ordine crescente di grado */

mostra(Polinomio) :-
    normalizza(Polinomio, Poli_norm),
    costruisci_coppie_grado_coeff(Poli_norm, Coppie_invertite),
    formatta termini(Coppie_invertite, primo).

/* Il predicato ausiliario costruisci_coppie_grado_coeff costruisce,
   tramite accumula_coppie, le coppie (grado, coefficiente) ordinate
   dal grado massimo al minimo:
   - il primo argomento è la lista dei coefficienti in ordine
     crescente di grado
   - il secondo argomento è la lista delle coppie risultante */

costruisci_coppie_grado_coeff(Polinomio, Coppie_invertite) :-
    accumula_coppie(Polinomio, 0, Coppie),
    reverse(Coppie, Coppie_invertite).

/* Il predicato ausiliario accumula_coppie associa ad ogni
   coefficiente il proprio grado, percorrendo il polinomio in ordine
   crescente a partire dal grado 0:
   - il primo argomento è la lista dei coefficienti residui da
     associare al grado
   - il secondo argomento è il grado corrente, incrementato a ogni
```

```

    passo della ricorsione
    - il terzo argomento è la lista delle coppie risultante */

accumula_coppie([], _, []).
accumula_coppie([Coeff|Resto], Grado, [(Grado, Coeff)|Coppie_resto]) :-
    Grado_succ is Grado + 1,
    accumula_coppie(Resto, Grado_succ, Coppie_resto).

/* Il predicato ausiliario formatta_termini stampa ricorsivamente i
termini di un polinomio, dal grado massimo al minimo, saltando i
coefficienti nulli:
- il primo argomento è la lista delle coppie (grado,
coefficiente) residue da stampare, ordinata dal grado massimo al
minimo
- il secondo argomento è l'atomo primo se il termine da stampare è
il primo termine non nullo del polinomio, l'atomo successivo
altrimenti */

formatta_termini([], primo) :- !, write('0').
formatta_termini([], successivo) :- !.
formatta_termini([(_, Coefficiente)|Resto], Posizione) :-
    tolleranza_numerica(T), abs(Coefficiente) < T, !,
    formatta_termini(Resto, Posizione).
formatta_termini([(Grado, Coefficiente)|Resto], Posizione) :-
    formatta_segno(Coefficiente, Posizione),
    Valore_assoluto is abs(Coefficiente),
    formatta_monomio(Grado, Valore_assoluto),
    formatta_termini(Resto, successivo).

/* Il predicato ausiliario formatta_segno stampa il segno da
anteporre al termine corrente:
- il primo argomento è il coefficiente del termine corrente
- il secondo argomento è l'atomo primo se il termine è il primo
termine non nullo del polinomio, l'atomo successivo altrimenti */

formatta_segno(Coefficiente, primo) :- Coefficiente < 0, !, write('-').
formatta_segno(_, primo) :- !.
formatta_segno(Coefficiente, successivo) :-
    Coefficiente < 0, !, write(' - ').
formatta_segno(_, successivo) :- write(' + ').

/* Il predicato ausiliario formatta_monomio stampa grado e
coefficiente di un monomio, omettendo i coefficienti unitari e le
potenze 0 e 1:
- il primo argomento è il grado del monomio
- il secondo argomento è il valore assoluto del coefficiente del
monomio */

formatta_monomio(0, Coefficiente) :- !, scrivi_valore(Coefficiente).
formatta_monomio(1, Coefficiente) :-
    tolleranza_numerica(T), abs(Coefficiente - 1.0) < T, !, write('x').
formatta_monomio(1, Coefficiente) :- !,
    scrivi_valore(Coefficiente), write('x').
formatta_monomio(Grado, Coefficiente) :-
    tolleranza_numerica(T), abs(Coefficiente - 1.0) < T, !,
    write('x^'), write(Grado).
formatta_monomio(Grado, Coefficiente) :-

```

```

    scrivi_valore(Coefficiente), write('x^'), write(Grado).

/* Il predicato ausiliario scrivi_valore stampa un coefficiente non
negativo come intero se la sua parte decimale è trascurabile,
altrimenti arrotondato a quattro cifre decimali e privato degli
zeri finali; i valori di modulo troppo grande per essere
arrotondati a intero vengono stampati nella forma nativa:
- il suo unico argomento è il valore del coefficiente da stampare */

scrivi_valore(Coefficiente) :-
    abs(Coefficiente) > 1.0e14, !, write(Coefficiente).
scrivi_valore(Coefficiente) :-
    tolleranza_numerica(T),
    Arrotondato is round(Coefficiente),
    abs(Coefficiente - Arrotondato) < T, !,
    write(Arrotondato).
scrivi_valore(Coefficiente) :-
    Parte_intera is truncate(Coefficiente),
    Frazione is Coefficiente - Parte_intera,
    Parte_frazionaria is round(Frazione * 10000),
    scrivi_parti(Parte_intera, Parte_frazionaria).

/* Il predicato ausiliario scrivi_parti stampa la parte intera e, se
significativa, la parte frazionaria di un coefficiente, gestendo
il riporto prodotto dall'arrotondamento alla quarta cifra
decimale:
- il primo argomento è la parte intera del coefficiente
- il secondo argomento è la parte frazionaria arrotondata,
  espressa come intero su quattro cifre */

scrivi_parti(Parte_intera, 10000) :- !,
    Parte_intera_succ is Parte_intera + 1,
    write(Parte_intera_succ).
scrivi_parti(Parte_intera, 0) :- !,
    write(Parte_intera).
scrivi_parti(Parte_intera, Parte_frazionaria) :-
    write(Parte_intera), write('.'),
    scrivi_cifre_frazionarie(Parte_frazionaria, 1000).

/* Il predicato ausiliario scrivi_cifre_frazionarie stampa le cifre
della parte frazionaria, dalla più significativa alla meno
significativa, arrestandosi quando le cifre residue sono tutte
nulle:
- il primo argomento è la parte frazionaria residua, espressa come
  intero su quattro cifre
- il secondo argomento è il peso della cifra da stampare */

scrivi_cifre_frazionarie(0, _) :- !.
scrivi_cifre_frazionarie(Parte_frazionaria, Peso) :-
    Cifra is Parte_frazionaria // Peso,
    Resto is Parte_frazionaria mod Peso,
    Peso_successivo is Peso // 10,
    write(Cifra),
    scrivi_cifre_frazionarie(Resto, Peso_successivo).

/* Il predicato grado_polinomio calcola il grado di un polinomio come
la lunghezza del polinomio normalizzato meno uno, oppure 0 se il

```

```

    polinomio è nullo:
    - il primo argomento è la lista dei coefficienti in ordine
      crescente di grado
    - il secondo argomento è il grado calcolato */

grado_polinomio(Polinomio, Grado) :-
    normalizza(Polinomio, Poli_norm),
    length(Poli_norm, Lunghezza),
    Lunghezza > 0, !,
    Grado is Lunghezza - 1.
grado_polinomio(_, 0).

/* Il predicato somma calcola la somma di due polinomi in tempo
   lineare O(n):
   - il primo argomento è la lista dei coefficienti del primo
     polinomio
   - il secondo argomento è la lista dei coefficienti del secondo
     polinomio
   - il terzo argomento è la lista dei coefficienti del polinomio
     somma, non normalizzata */

somma([], Secondo_poli, Secondo_poli) :- !.
somma(Primo_poli, [], Primo_poli) :- !.
somma([C1|Resto1], [C2|Resto2], [Somma|Somme_resto]) :-
    Somma is C1 + C2,
    somma(Resto1, Resto2, Somme_resto).

/* Il predicato differenza calcola la differenza tra due polinomi in
   tempo lineare O(n):
   - il primo argomento è la lista dei coefficienti del primo
     polinomio
   - il secondo argomento è la lista dei coefficienti del secondo
     polinomio
   - il terzo argomento è la lista dei coefficienti del polinomio
     differenza, non normalizzata */

differenza([], Secondo_poli, Risultato) :- !,
    moltiplica_per_scalare(Secondo_poli, -1, Risultato).
differenza(Primo_poli, [], Primo_poli) :- !.
differenza([C1|Resto1], [C2|Resto2], [Diff|Diff_resto]) :-
    Diff is C1 - C2,
    differenza(Resto1, Resto2, Diff_resto).

/* Il predicato ausiliario moltiplica_per_scalare moltiplica ogni
   coefficiente di una lista per uno scalare:
   - il primo argomento è la lista dei coefficienti da moltiplicare
   - il secondo argomento è lo scalare per cui moltiplicare ogni
     coefficiente
   - il terzo argomento è la lista dei coefficienti risultante */

moltiplica_per_scalare([], _, []).
moltiplica_per_scalare([Y|Y_resto], Scalare, [Prod|Prod_resto]) :-
    Prod is Y * Scalare,
    moltiplica_per_scalare(Y_resto, Scalare, Prod_resto).

/* Il predicato prodotto calcola il prodotto di due polinomi:
   - il primo argomento è la lista dei coefficienti del primo

```

```

    polinomio
- il secondo argomento è la lista dei coefficienti del secondo
  polinomio
- il terzo argomento è la lista dei coefficienti del polinomio
  prodotto, non normalizzata */

prodotto([], _, []) :- !.
prodotto(_, [], []) :- !.
prodotto([Coeff_testa|Coeff_resto], Secondo_poli, Prodotto) :-
    moltiplica_per_scalare(Secondo_poli, Coeff_testa, Prodotto_testa),
    prodotto(Coeff_resto, Secondo_poli, Prodotto_resto),
    somma(Prodotto_testa, [0.0|Prodotto_resto], Prodotto).

/* Il predicato gestisci_divisione coordina la divisione euclidea tra
   due polinomi e la stampa del quoziente e del resto, segnalando
   l'impossibilità di dividere per il polinomio nullo, caso nel quale
   il predicato divisione fallisce:
- il primo argomento è la lista dei coefficienti del polinomio
  dividendo
- il secondo argomento è la lista dei coefficienti del polinomio
  divisore */

gestisci_divisione(Dividendo, Divisore) :-
    divisione(Dividendo, Divisore, Quoziente, Resto), !,
    write('Quoziente:      '), mostra(Quoziente), nl,
    write('Resto:          '), mostra(Resto), nl.
gestisci_divisione(_, _) :-
    write('Errore:          impossibile dividere per '),
    write('il polinomio nullo. '), nl.

/* Il predicato divisione calcola quoziente e resto della divisione
   euclidea tra due polinomi, fallendo se il divisore è il polinomio
   nullo:
- il primo argomento è la lista dei coefficienti del polinomio
  dividendo
- il secondo argomento è la lista dei coefficienti del polinomio
  divisore
- il terzo argomento è la lista dei coefficienti del polinomio
  quoziente
- il quarto argomento è la lista dei coefficienti del polinomio
  resto */

divisione(Dividendo, Divisore, Quoziente, Resto) :-
    normalizza(Divisore, Divisore_normalizzato),
    Divisore_normalizzato \== [], !,
    normalizza(Dividendo, Dividendo_normalizzato),
    divisione_ricorsiva(Dividendo_normalizzato,
                        Divisore_normalizzato,
                        [],
                        Quoziente,
                        Resto).

/* Il predicato ausiliario divisione_ricorsiva esegue la divisione
   lunga accumulando il quoziente, delegando a passo_divisione la
   scelta tra caso base e caso generale in base alle lunghezze delle
   due liste, calcolate una sola volta ad ogni passo:
- il primo argomento è la lista dei coefficienti del dividendo

```



```

    corrente, normalizzata
- il secondo argomento è la lista dei coefficienti del divisore,
  normalizzata
- il terzo argomento è la lista dei coefficienti del quoziente
  parziale accumulato finora
- il quarto argomento è la lista dei coefficienti del quoziente
  finale risultante
- il quinto argomento è la lista dei coefficienti del resto finale
  risultante */

divisione_ricorsiva(Dividendo_corrente,
                    Divisore_normalizzato,
                    Quoziente_parziale,
                    Quoziente,
                    Resto) :-
    length(Dividendo_corrente, Lungh_dividendo),
    length(Divisore_normalizzato, Lungh_divisore),
    passo_divisione(Lungh_dividendo,
                    Lungh_divisore,
                    Dividendo_corrente,
                    Divisore_normalizzato,
                    Quoziente_parziale,
                    Quoziente,
                    Resto).

/* Il predicato ausiliario passo_divisione conclude la divisione se il
   grado del dividendo corrente è inferiore a quello del divisore,
   altrimenti esegue un passo di divisione lunga e prosegue
   ricorsivamente:
- il primo argomento è la lunghezza della lista del dividendo
  corrente
- il secondo argomento è la lunghezza della lista del divisore
- il terzo argomento è la lista dei coefficienti del dividendo
  corrente, normalizzata
- il quarto argomento è la lista dei coefficienti del divisore,
  normalizzata
- il quinto argomento è la lista dei coefficienti del quoziente
  parziale accumulato finora
- il sesto argomento è la lista dei coefficienti del quoziente
  finale risultante
- il settimo argomento è la lista dei coefficienti del resto finale
  risultante */

passo_divisione(Lungh_dividendo,
                Lungh_divisore,
                Dividendo_corrente,
                -,
                Quoziente_parziale,
                Quoziente,
                Resto) :-
    Lungh_dividendo < Lungh_divisore,
    !,
    normalizza(Quoziente_parziale, Quoziente),
    normalizza(Dividendo_corrente, Resto).
passo_divisione(Lungh_dividendo,
                Lungh_divisore,
                Dividendo_corrente,

```

```

        Divisore_normalizzato,
        Quoziente_parziale,
        Quoziente,
        Resto) :-
last(Dividendo_corrente, Direttore_dividendo),
last(Divisore_normalizzato, Direttore_divisore),
Coefficiente_termine is float(Direttore_dividendo) /
        float(Direttore_divisore),
Differenza_grado is Lungh_dividendo - Lungh_divisore,
costruisci_monomio(Differenza_grado,
        Coefficiente_termine,
        Termine_quoziente),
prodotto(Divisore_normalizzato,
        Termine_quoziente,
        Termine_da_sottrarre),
differenza(Dividendo_corrente,
        Termine_da_sottrarre,
        Dividendo_aggiornato),
somma(Quoziente_parziale, Termine_quoziente, Quoziente_aggiornato),
normalizza(Dividendo_aggiornato, Dividendo_aggiornato_norm),
divisione_ricorsiva(Dividendo_aggiornato_norm,
        Divisore_normalizzato,
        Quoziente_aggiornato,
        Quoziente,
        Resto).

/* Il predicato ausiliario costruisci_monomio genera una lista di
coefficienti, con zeri nei gradi inferiori, rappresentante un
unico monomio:
- il primo argomento è il grado del monomio da costruire
- il secondo argomento è il coefficiente del monomio
- il terzo argomento è la lista dei coefficienti risultante,
rappresentante il monomio */

costruisci_monomio(0, Coefficiente, [Coefficiente]) :- !.
costruisci_monomio(N, Coefficiente, [0.0|Resto]) :-
    N > 0, N_1 is N - 1,
    costruisci_monomio(N_1, Coefficiente, Resto).

/* Il predicato mcd calcola il massimo comune divisore di due polinomi
tramite l'algoritmo di Euclide, restituendo il risultato reso
monico:
- il primo argomento è la lista dei coefficienti del primo
polinomio
- il secondo argomento è la lista dei coefficienti del secondo
polinomio
- il terzo argomento è la lista dei coefficienti del massimo comun
divisore risultante */

mcd(Primo_poli, Secondo_poli, Mcd) :-
    normalizza(Primo_poli, Primo_poli_norm),
    normalizza(Secondo_poli, Secondo_poli_norm),
    euclide(Primo_poli_norm, Secondo_poli_norm, Mcd).

/* Il predicato ausiliario euclide applica ricorsivamente
l'algoritmo di Euclide ai due polinomi, tramite monico, fino
a ridurre il secondo polinomio al polinomio nullo:

```

```
- il primo argomento è la lista dei coefficienti del primo
  polinomio, normalizzata
- il secondo argomento è la lista dei coefficienti del secondo
  polinomio, normalizzata
- il terzo argomento è la lista dei coefficienti del massimo comun
  divisore risultante */

euclide(Primo_poli, [], Mcd) :- !,
    monico(Primo_poli, Mcd).
euclide(Primo_poli, Secondo_poli, Mcd) :-
    divisione(Primo_poli, Secondo_poli, _, Resto),
    normalizza(Resto, Resto_norm),
    euclide(Secondo_poli, Resto_norm, Mcd).

/* Il predicato ausiliario monico divide tutti i coefficienti
  di un polinomio per il suo coefficiente direttore (l'ultimo della
  lista), tramite moltiplica_per_scalare:
- il primo argomento è la lista dei coefficienti da rendere monica
- il secondo argomento è la lista dei coefficienti resa monica
  risultante */

monico(Coefficienti, Monico) :-
    normalizza(Coefficienti, Normalizzati),
    monico_normalizzato(Normalizzati, Monico).

/* Il predicato ausiliario monico_normalizzato distingue il
  caso del polinomio nullo, per il quale il risultato è il polinomio
  nullo stesso, dal caso generale:
- il primo argomento è la lista dei coefficienti da rendere
  monica, già normalizzata
- il secondo argomento è la lista dei coefficienti resa monica
  risultante */

monico_normalizzato([], []) :- !.
monico_normalizzato(Coefficienti, Monico) :-
    last(Coefficienti, Coeff_direttore),
    Inverso_coefficiente is 1.0 / Coeff_direttore,
    moltiplica_per_scalare(Coefficienti, Inverso_coefficiente, Monico).
```

5 Testing del Programma

5.1 Testing del programma Haskell

Test Haskell 1

```
Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 0
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 0
```

```
Polinomio A:      0
Polinomio B:      0
Grado A:          0
Grado B:          0
Somma:            0
Differenza:       0
Prodotto:         0
Errore:           impossibile dividere per il polinomio nullo.
MCD:              0
```

Test Haskell 2

```
Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 0
```

```
Polinomio A:      1
Polinomio B:      0
Grado A:          0
Grado B:          0
Somma:            1
Differenza:       1
Prodotto:         0
Errore:           impossibile dividere per il polinomio nullo.
MCD:              1
```

Test Haskell 3

```
Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 0
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 1
```

```
Polinomio A:      0
Polinomio B:      1
Grado A:          0
Grado B:          0
Somma:            1
Differenza:       -1
Prodotto:         0
Quoziente:        0
Resto:            0
MCD:              1
```

Test Haskell 4

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 2

Polinomio A: 1
Polinomio B: 2
Grado A: 0
Grado B: 0
Somma: 3
Differenza: -1
Prodotto: 2
Quoziente: 0.5
Resto: 0
MCD: 1

Test Haskell 5

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 3 4 5

Polinomio A: $3x^2 + 2x + 1$
Polinomio B: $5x^2 + 4x + 3$
Grado A: 2
Grado B: 2
Somma: $8x^2 + 6x + 4$
Differenza: $-2x^2 - 2x - 2$
Prodotto: $15x^4 + 22x^3 + 22x^2 + 10x + 3$
Quoziente: 0.6
Resto: $-0.4x - 0.8$
MCD: 1

Test Haskell 6

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 3 2 1

Polinomio A: $3x^2 + 2x + 1$
Polinomio B: $x^2 + 2x + 3$
Grado A: 2
Grado B: 2
Somma: $4x^2 + 4x + 4$
Differenza: $2x^2 - 2$
Prodotto: $3x^4 + 8x^3 + 14x^2 + 8x + 3$
Quoziente: 3
Resto: $-4x - 8$
MCD: 1

Test Haskell 7

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 6 5 4

```
Polinomio A:      3x^2 + 2x + 1
Polinomio B:      4x^2 + 5x + 6
Grado A:          2
Grado B:          2
Somma:            7x^2 + 7x + 7
Differenza:       -x^2 - 3x - 5
Prodotto:         12x^4 + 23x^3 + 32x^2 + 17x + 6
Quoziente:        0.75
Resto:            -1.75x - 3.5
MCD:              1
```

Test Haskell 8

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 2 3 0

```
Polinomio A:      3x^2 + 2x + 1
Polinomio B:      3x + 2
Grado A:          2
Grado B:          1
Somma:            3x^2 + 5x + 3
Differenza:       3x^2 - x - 1
Prodotto:         9x^3 + 12x^2 + 7x + 2
Quoziente:        x
Resto:            1
MCD:              1
```

Test Haskell 9

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 0

```
Polinomio A:      3x^2 + 2x + 1
Polinomio B:      0
Grado A:          2
Grado B:          0
Somma:            3x^2 + 2x + 1
Differenza:       3x^2 + 2x + 1
Prodotto:         0
Errore:           impossibile dividere per il polinomio nullo.
MCD:              x^2 + 0.6667x + 0.3333
```

Test Haskell 10

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 2 -3 1
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 3 -4 1

```
Polinomio A:      x^2 - 3x + 2
Polinomio B:      x^2 - 4x + 3
Grado A:          2
Grado B:          2
Somma:            2x^2 - 7x + 5
Differenza:       x - 1
Prodotto:         x^4 - 7x^3 + 17x^2 - 17x + 6
Quoziente:        1
Resto:            x - 1
MCD:              x - 1
```

5.2 Testing del programma Prolog

Test Prolog 1

```
Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 0
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 0
```

```
Polinomio A:      0
Polinomio B:      0
Grado A:          0
Grado B:          0
Somma:            0
Differenza:       0
Prodotto:         0
Errore:           impossibile dividere per il polinomio nullo.
MCD:              0
```

yes

Test Prolog 2

```
Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 0
```

```
Polinomio A:      1
Polinomio B:      0
Grado A:          0
Grado B:          0
Somma:            1
Differenza:       1
Prodotto:         0
Errore:           impossibile dividere per il polinomio nullo.
MCD:              1
```

yes

Test Prolog 3

```
Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 0
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 1
```

```
Polinomio A:      0
Polinomio B:      1
Grado A:          0
Grado B:          0
Somma:            1
Differenza:      -1
Prodotto:         0
Quoziente:        0
Resto:            0
MCD:              1
```

yes

Test Prolog 4

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 2

```
Polinomio A:      1
Polinomio B:      2
Grado A:          0
Grado B:          0
Somma:            3
Differenza:       -1
Prodotto:         2
Quoziente:        0.5
Resto:            0
MCD:              1
```

yes

Test Prolog 5

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 3 4 5

```
Polinomio A:      3x^2 + 2x + 1
Polinomio B:      5x^2 + 4x + 3
Grado A:          2
Grado B:          2
Somma:            8x^2 + 6x + 4
Differenza:       -2x^2 - 2x - 2
Prodotto:         15x^4 + 22x^3 + 22x^2 + 10x + 3
Quoziente:        0.6
Resto:            -0.4x - 0.8
MCD:              1
```

yes

Test Prolog 6

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 3 2 1

```
Polinomio A:      3x^2 + 2x + 1
Polinomio B:      x^2 + 2x + 3
Grado A:          2
Grado B:          2
Somma:            4x^2 + 4x + 4
Differenza:       2x^2 - 2
Prodotto:         3x^4 + 8x^3 + 14x^2 + 8x + 3
Quoziente:        3
Resto:            -4x - 8
MCD:              1
```

yes

Test Prolog 7

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 6 5 4

```
Polinomio A:      3x^2 + 2x + 1
Polinomio B:      4x^2 + 5x + 6
Grado A:          2
Grado B:          2
Somma:            7x^2 + 7x + 7
Differenza:       -x^2 - 3x - 5
Prodotto:         12x^4 + 23x^3 + 32x^2 + 17x + 6
Quoziente:        0.75
Resto:            -1.75x - 3.5
MCD:              1
```

yes

Test Prolog 8

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 2 3 0

```
Polinomio A:      3x^2 + 2x + 1
Polinomio B:      3x + 2
Grado A:          2
Grado B:          1
Somma:            3x^2 + 5x + 3
Differenza:       3x^2 - x - 1
Prodotto:         9x^3 + 12x^2 + 7x + 2
Quoziente:        x
Resto:            1
MCD:              1
```

yes

Test Prolog 9

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 1 2 3
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 0

```
Polinomio A:      3x^2 + 2x + 1
Polinomio B:      0
Grado A:          2
Grado B:          0
Somma:            3x^2 + 2x + 1
Differenza:       3x^2 + 2x + 1
Prodotto:         0
Errore:           impossibile dividere per il polinomio nullo.
MCD:              x^2 + 0.6667x + 0.3333
```

yes

Test Prolog 10

Inserisci i coefficienti del polinomio A separati da spazi (ordine crescente di grado): 2 -3 1
Inserisci i coefficienti del polinomio B separati da spazi (ordine crescente di grado): 3 -4 1

```
Polinomio A:      x^2 - 3x + 2
Polinomio B:      x^2 - 4x + 3
Grado A:          2
Grado B:          2
Somma:            2x^2 - 7x + 5
Differenza:       x - 1
Prodotto:         x^4 - 7x^3 + 17x^2 - 17x + 6
Quoziente:        1
Resto:            x - 1
MCD:              x - 1
```

yes